# Trie

Theory & Application

# About author

- Full name: Lư Anh Khoa
- Email: luanhkhoa123@gmail.com
- Software Engineering at WorldQuant
- Graduated from HCMUT.
- Has some experience in competitive programming contests.

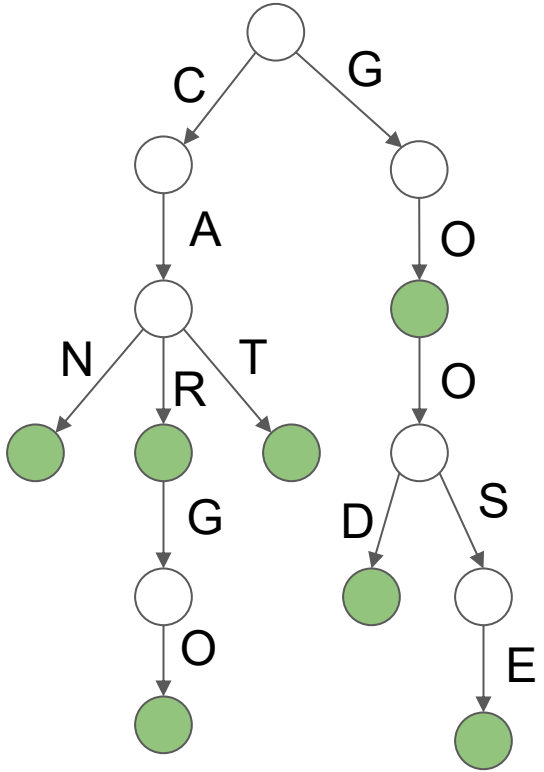
# Table of Contents

I. What is Trie ?
II. Application
III. Exercises

# I. What is Trie ?

# 1. Overview

- Come from the word “re**trie**val”.
- A tree data structure.
- Is also called **prefix tree**.
- Represents a set of strings.
- Can **insert**, **search**, **delete** a string S in O(|S|).



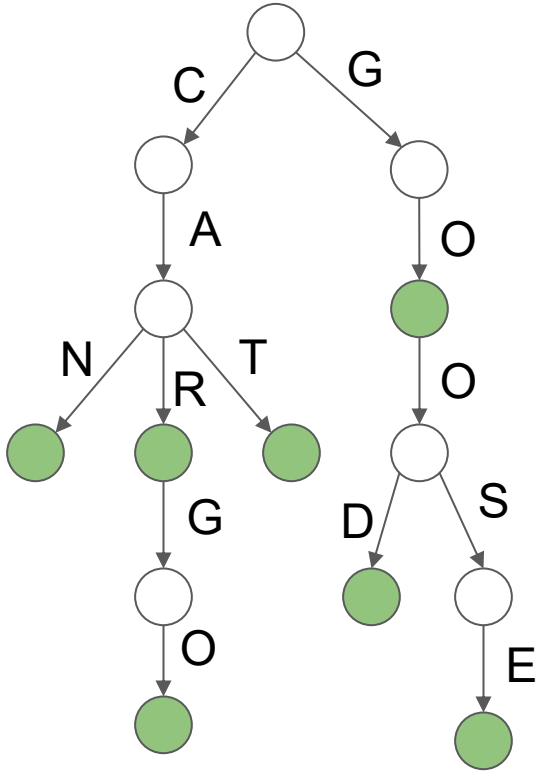

A trie represents the set {CAN, CAR, CARGO, CAT, GO, GOOD, GOOSE}
Green means end of word.

# 2. Pronunciation

- /ˈtriː/ (like “tree”).
- /ˈtraɪ/ (like “try”).
- tri - e (my Vietnamese pronunciation).

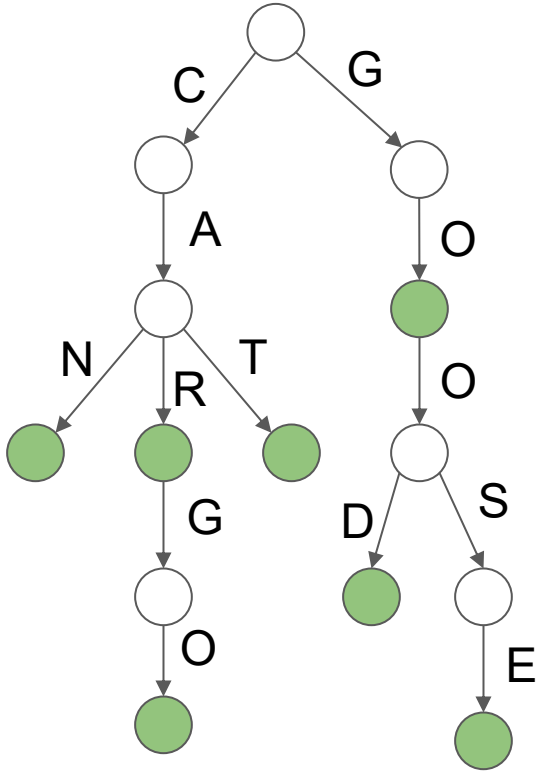
# 3. Implementation

- To implement a trie, we need to design 4 things:
  - Node
  - Insert
  - Search
  - Delete



# 3. Implementation - Node

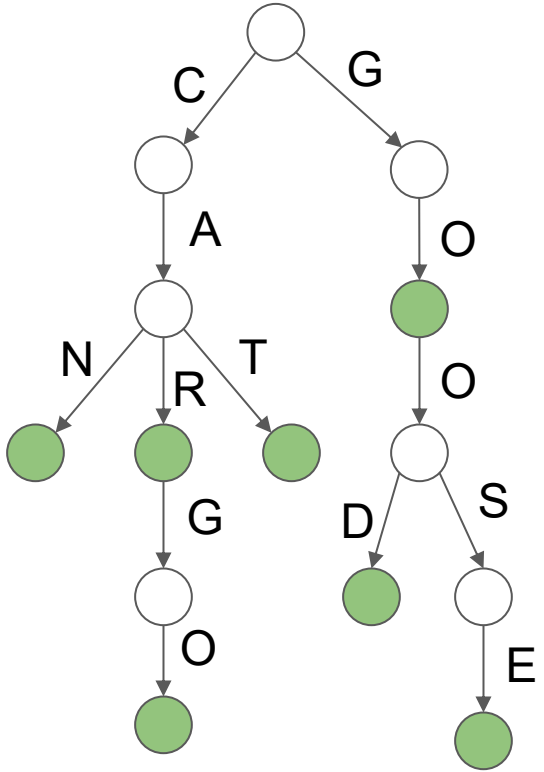
- Tries are composed of **nodes**.
- Each node contains links to other child nodes, or nil.
- Each node contains **R** links where **R** is the cardinality of the alphabet.
- Typically,
  **R** = 26 (english alphabet)
  or **R** = 256 (ASCII characters).



# 3. Implementation - Node

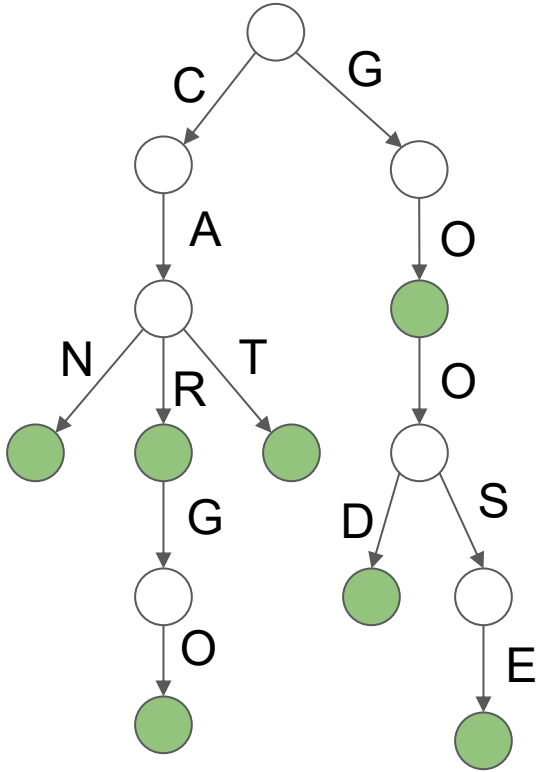
```
struct Node {
   NodeRef children[MAX_ALPHABET];
   boolean is_terminal;
}
```

trie_root = **Node()**



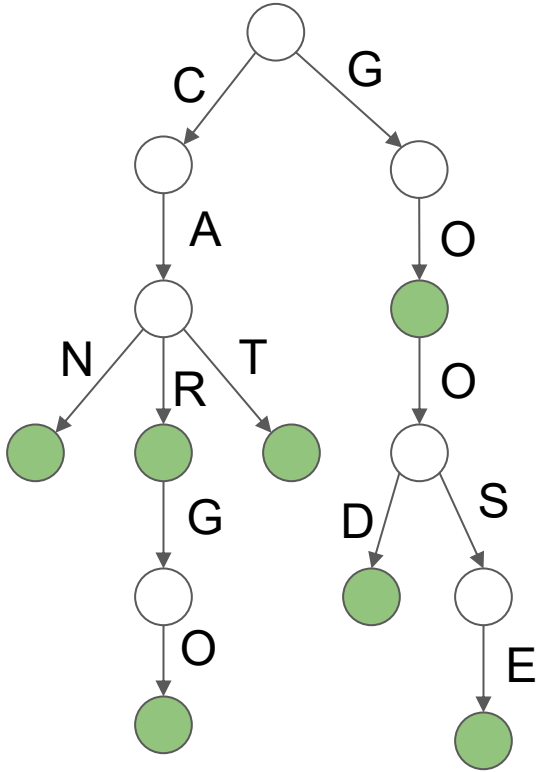
# 3. Implementation - Insert

- Insertion into trie is guided by using the character sets as the indexes into the children array until last character of the string key is reached.



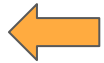
# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```



# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

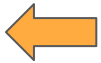


insert(root, “CAR”)



# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```



c == ‘C’

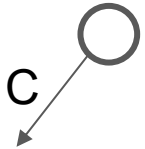


# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```



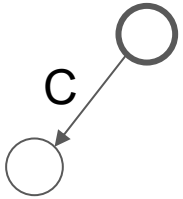
No node at ‘C’



# 3. Implementation - Insert

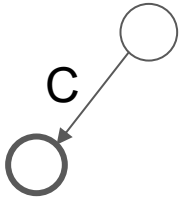
```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

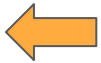
Create new Node



# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

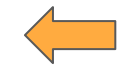


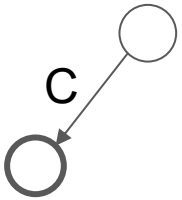
Go to child

# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```



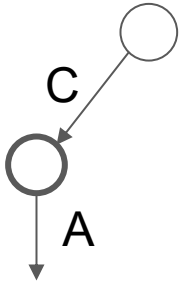
c == ‘A’



# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

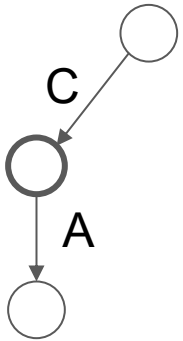
No node at ‘A’



# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

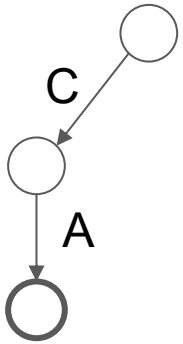
Create new Node



# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

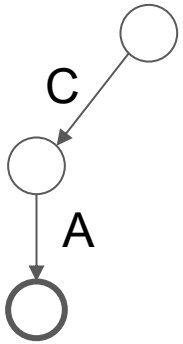


# 3. Implementation - Insert

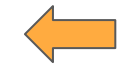
```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```



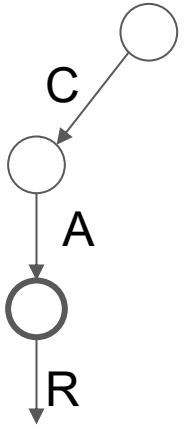



# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```



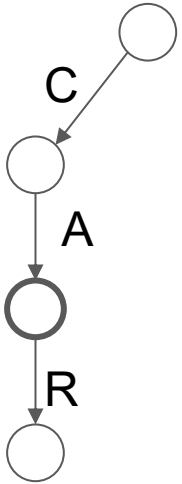
No node at ‘R’



# 3. Implementation - Insert

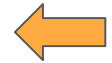
```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

Create new Node

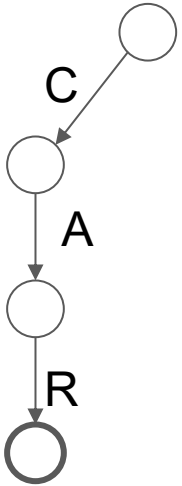


# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

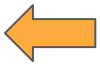


Go to child

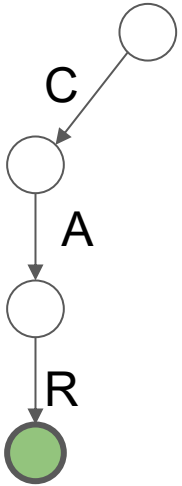


# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```



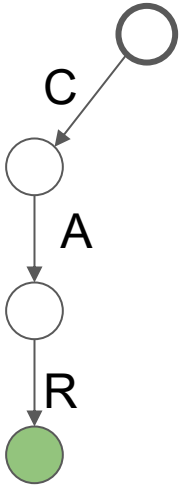
Mark as terminal



# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

insert(root, “CARGO”)

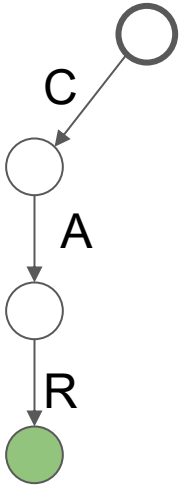


# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

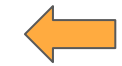


c == ‘C’

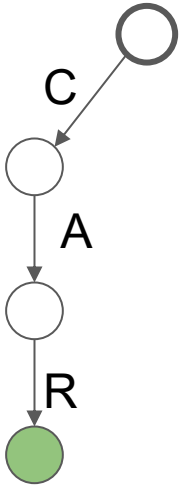


# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

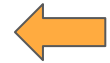


Have node at ‘C’

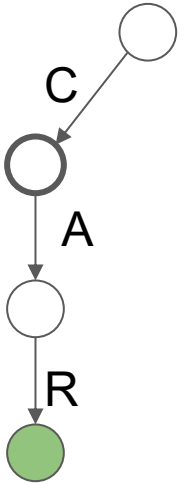


# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```



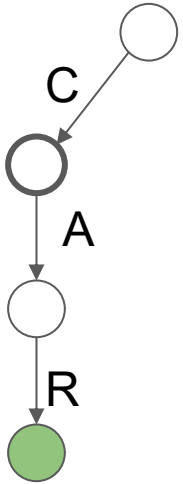
Go to child



# 3. Implementation - Insert

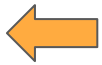
```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```



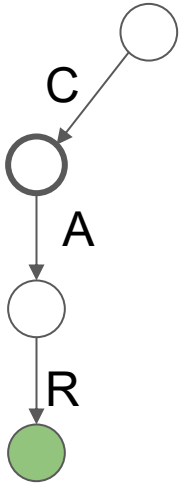



# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

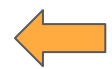


Has node at ‘A’

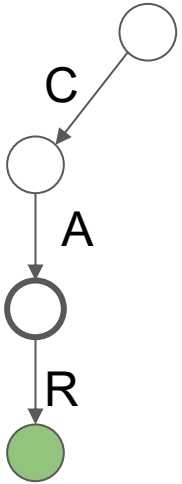


# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```



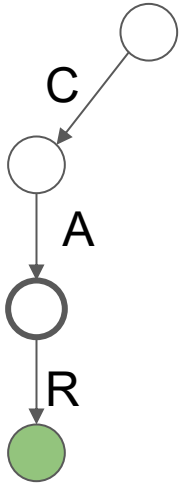
Go to child



# 3. Implementation - Insert

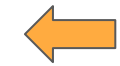
```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

c == ‘R’

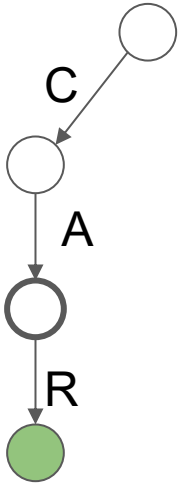


# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

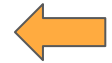


Has node at ‘R’

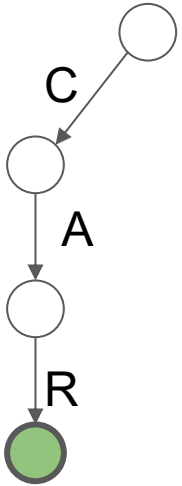


# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```



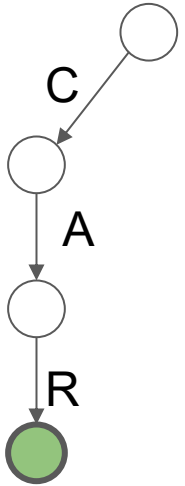
Go to child



# 3. Implementation - Insert

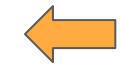
```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

c == ‘G’

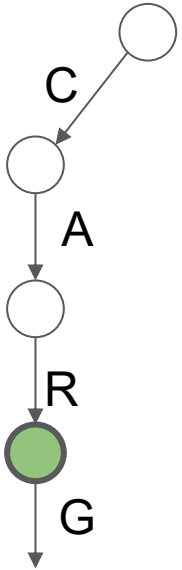


# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```



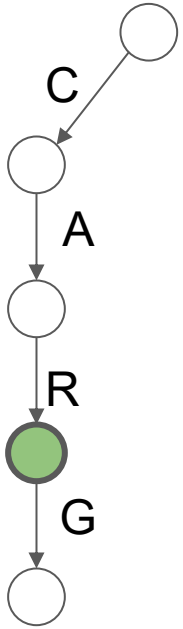
No node at ‘G’



# 3. Implementation - Insert

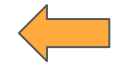
```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

Create new Node

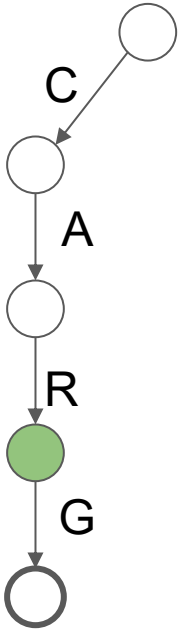


# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```



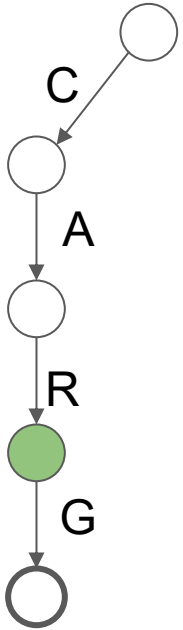
Go to child



# 3. Implementation - Insert

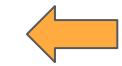
```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

c == ‘O’

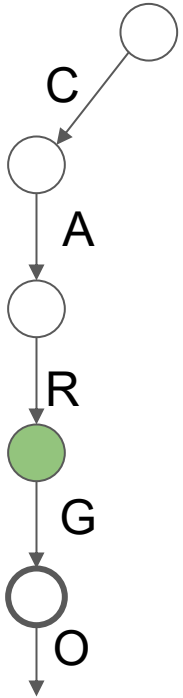


# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```



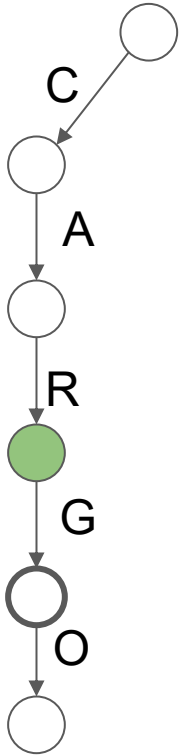
No node at ‘O’



# 3. Implementation - Insert

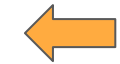
```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

Create new Node

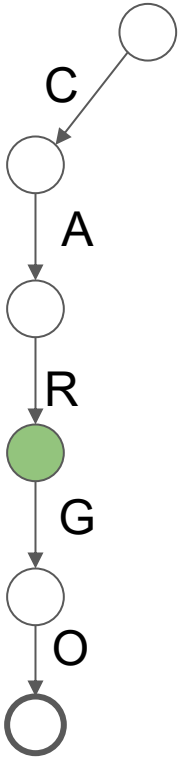


# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

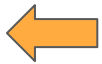


Go to child

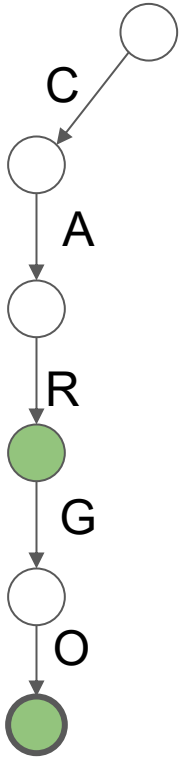


# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```



Mark as terminal

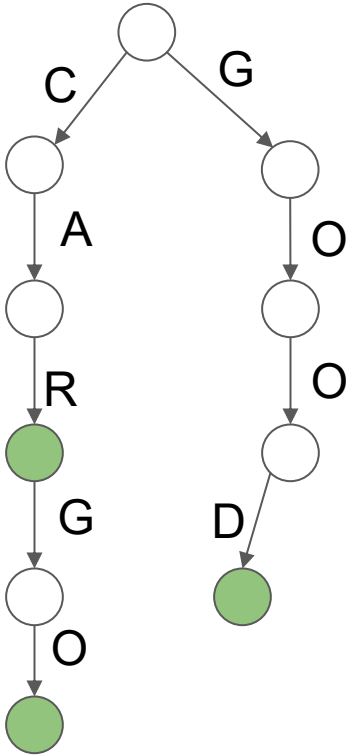


# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```



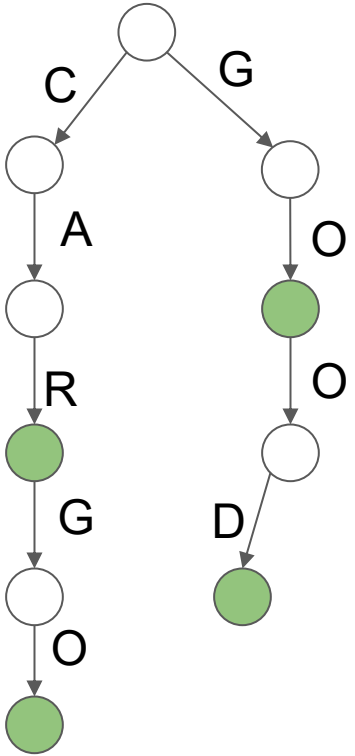
insert(root, “GOOD”)



# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

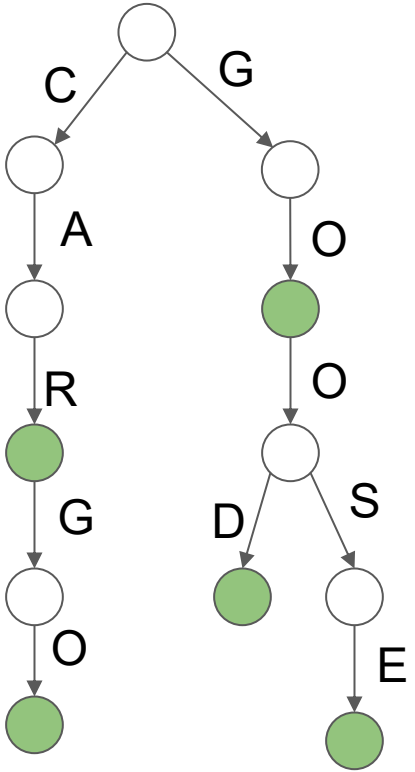
insert(root, “GO”)



# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

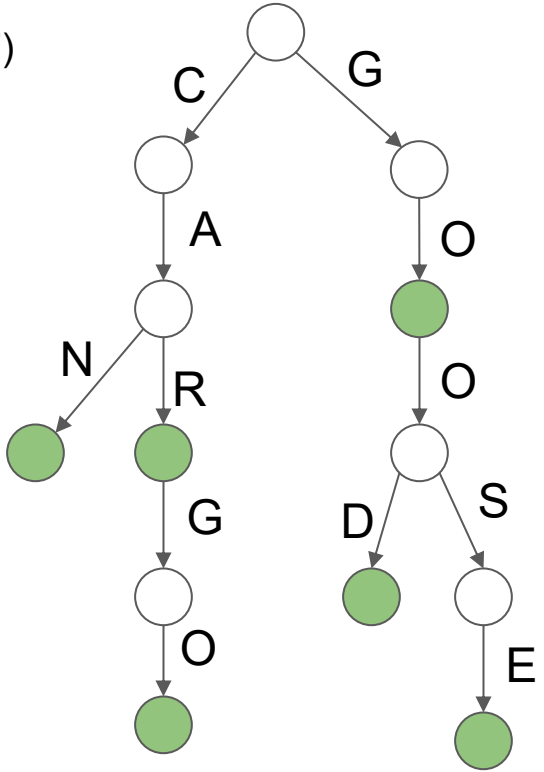
insert(root, “GOOSE”)



# 3. Implementation - Insert

```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

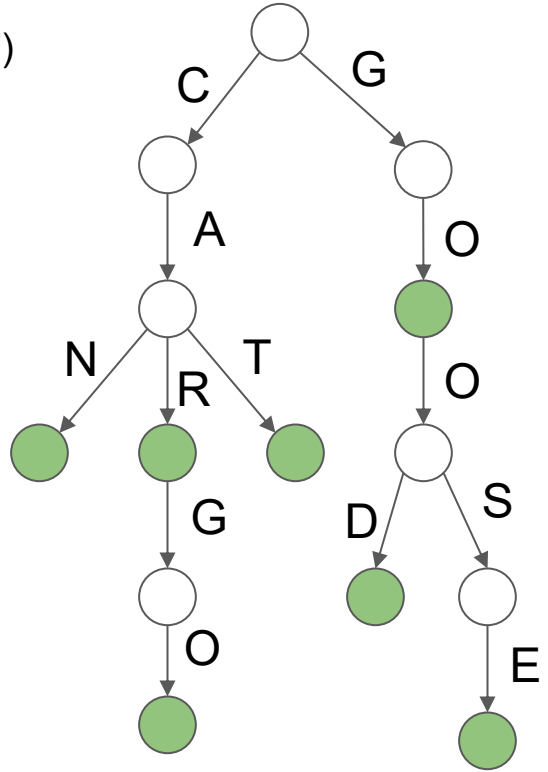
insert(root, “CAN”)



# 3. Implementation - Insert

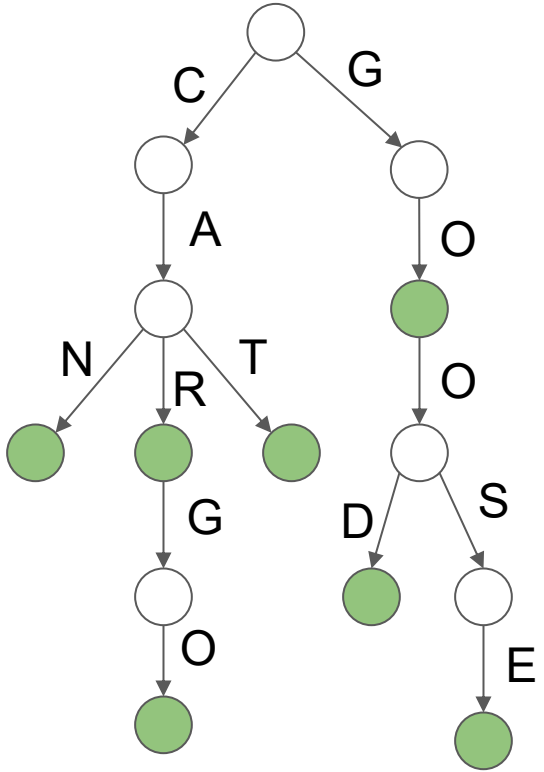
```
def insert(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      node.children[c] = Node()
    end if
    node = node.children[c]
  end for
  node.is_terminal = True
```

insert(root, “CAT”)



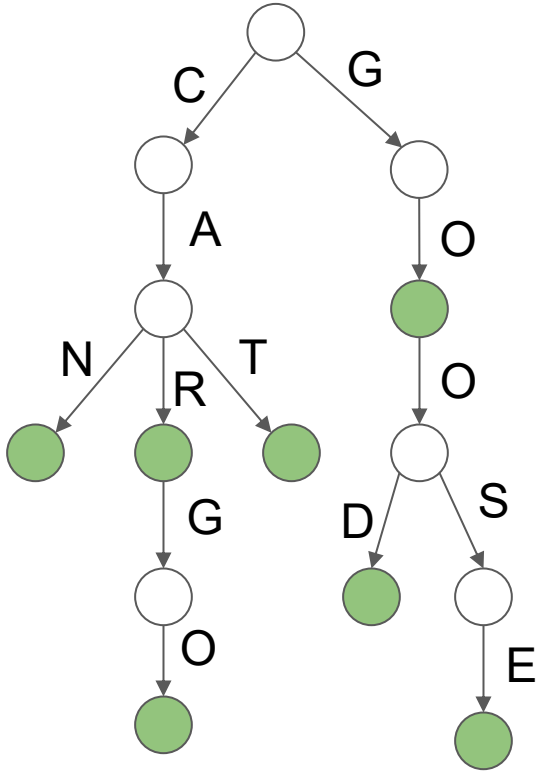
# 3. Implementation - Search

- Following the string within the trie.



# 3. Implementation - Search

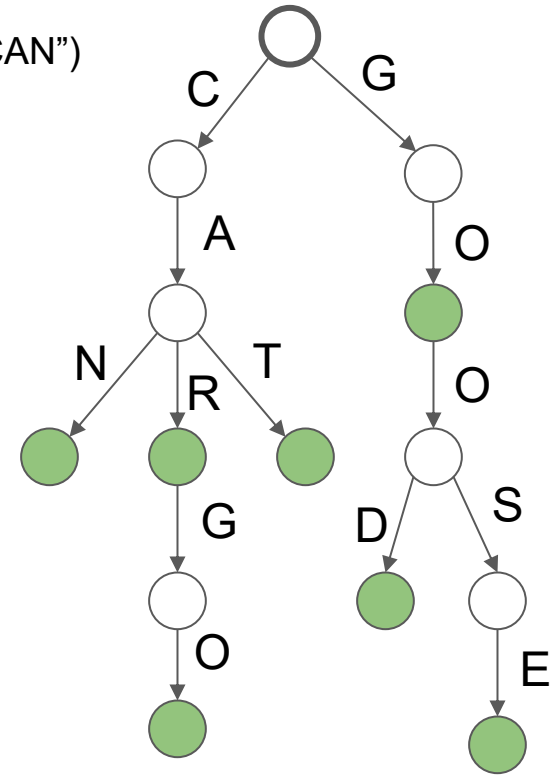
```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```



# 3. Implementation - Search

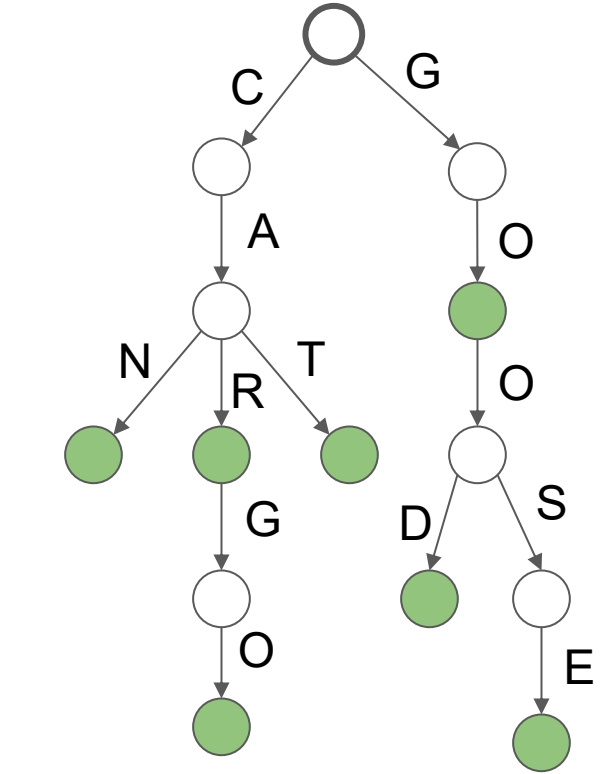
```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```

search(root, “CAN”)



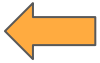
# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```

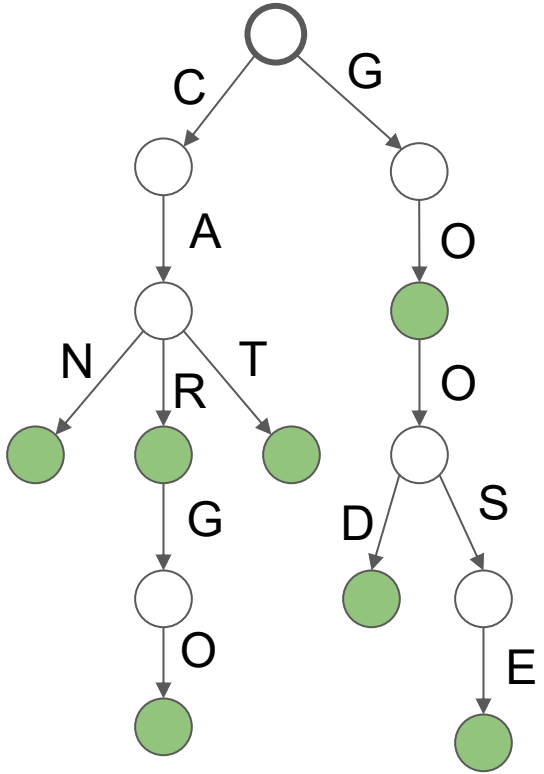


# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```



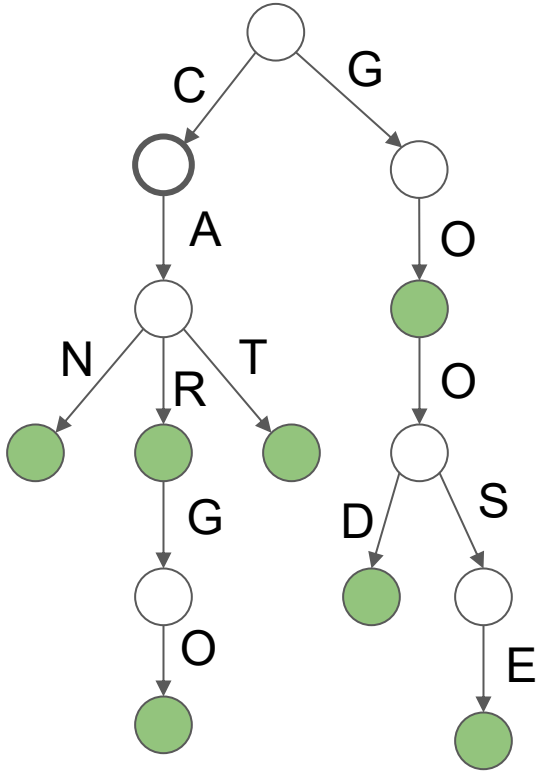
Has node at ‘C’



# 3. Implementation - Search

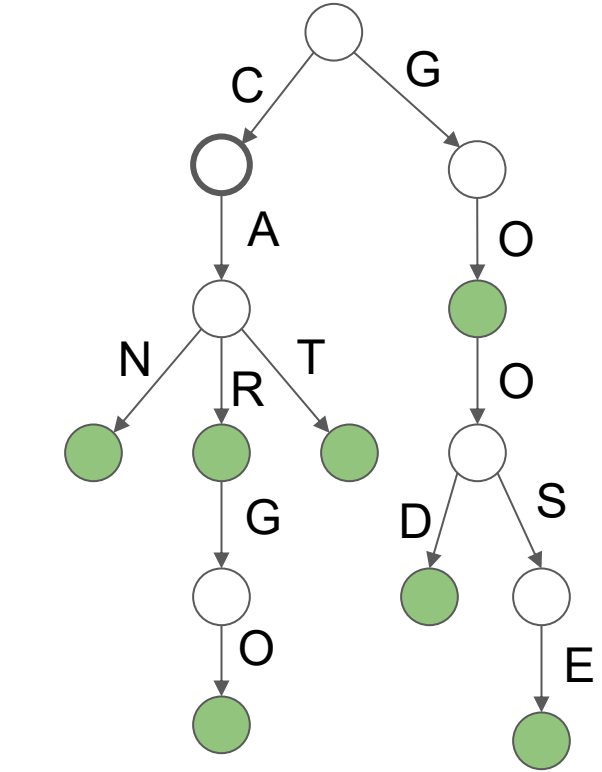
```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```

Go to child



# 3. Implementation - Search

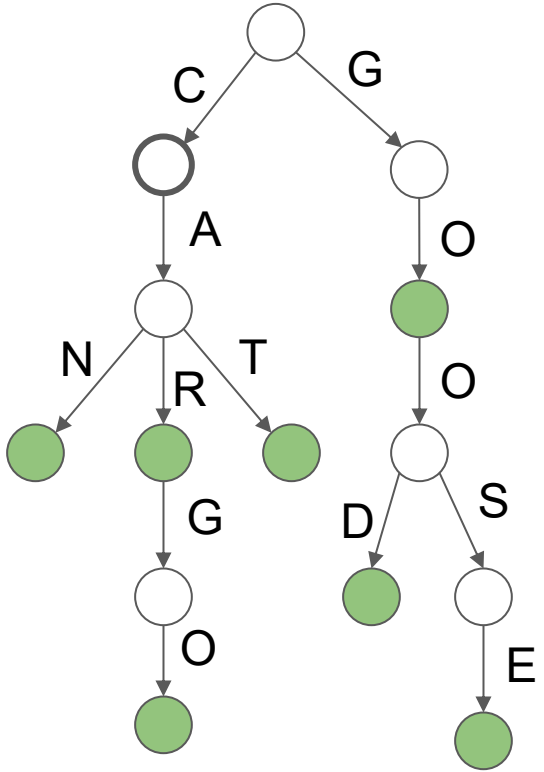
```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```



# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```

Has node at ‘A’

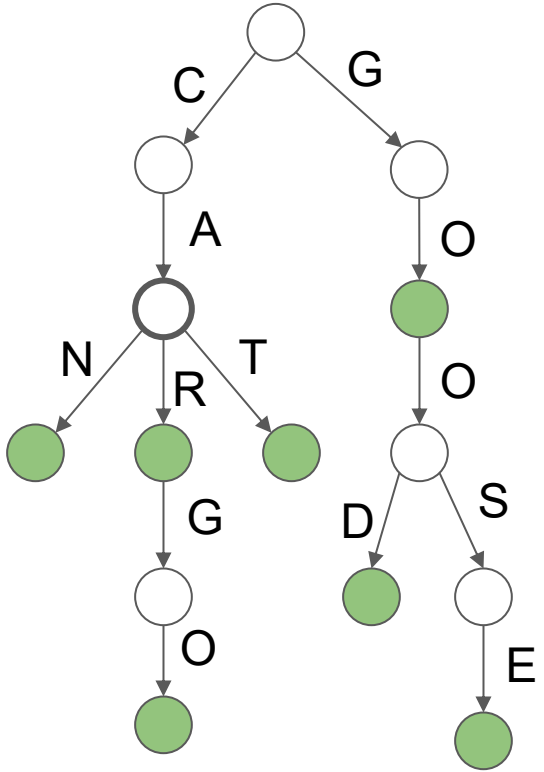


# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```

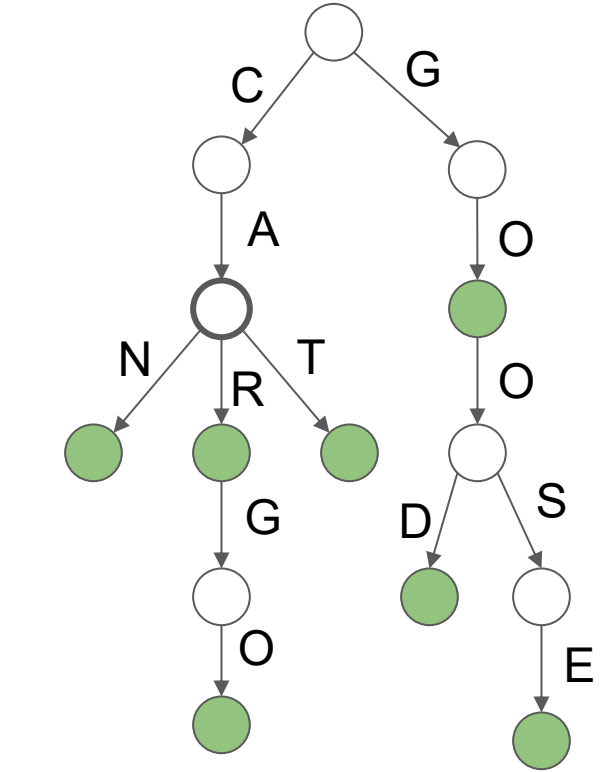


Go to child



# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```

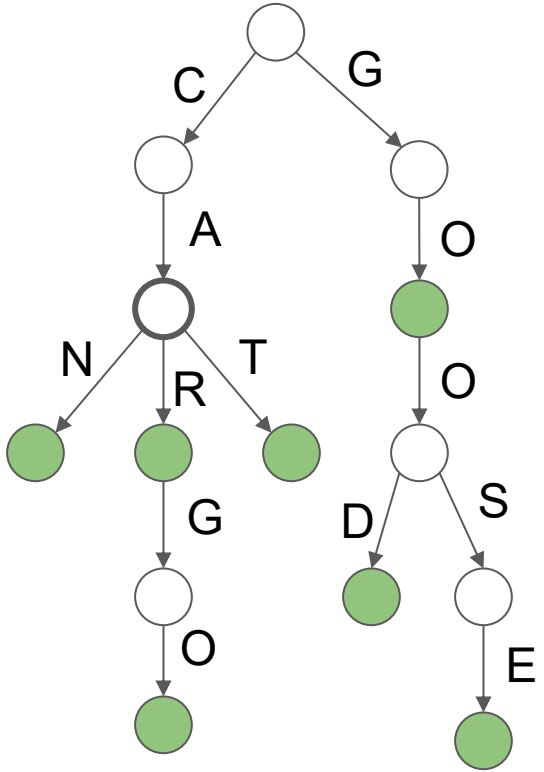


# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```

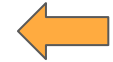


Has node at ‘N’

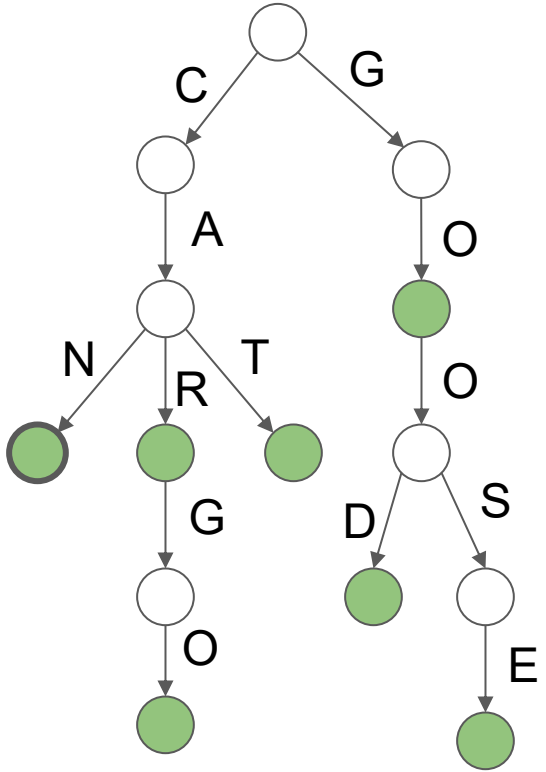


# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```

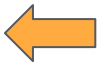


Go to child

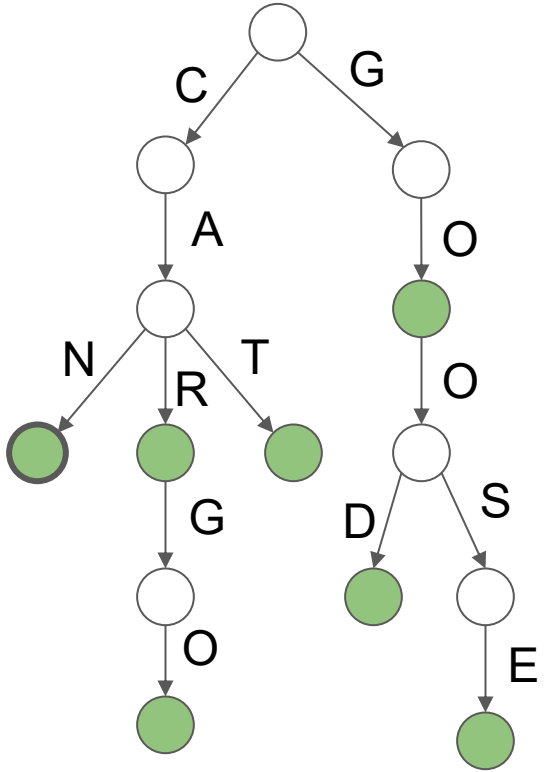


# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```



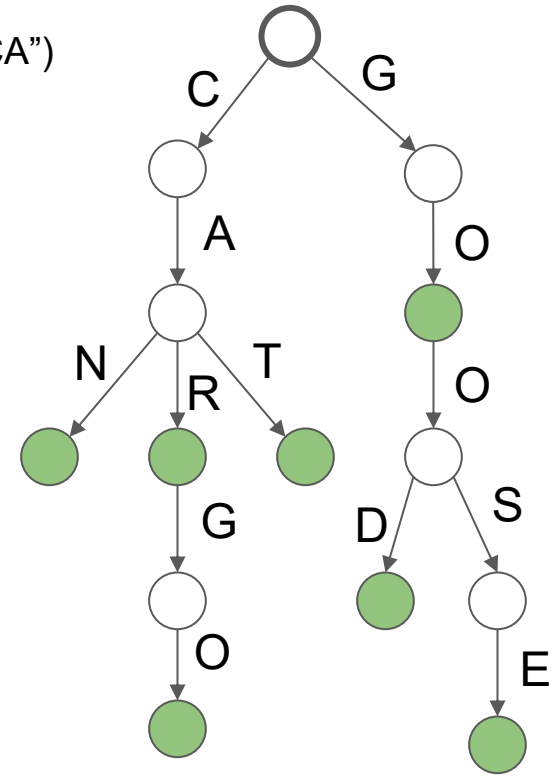
True



# 3. Implementation - Search

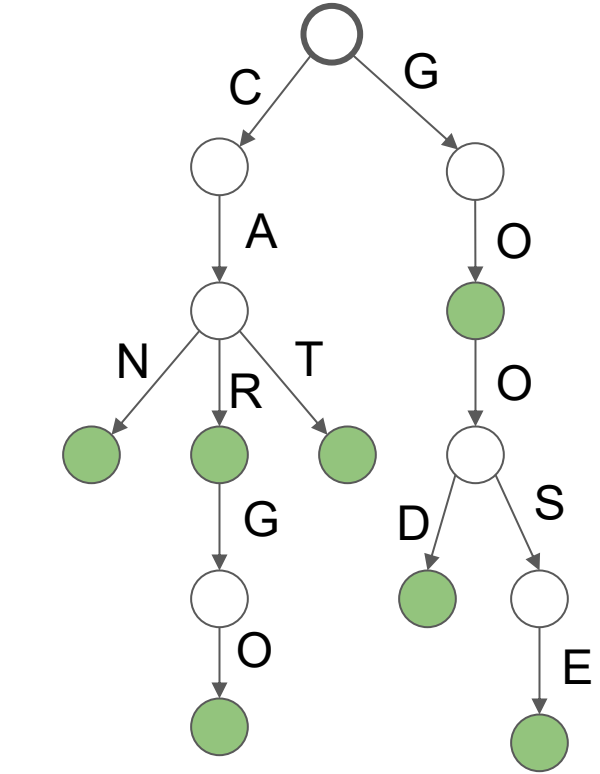
```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```

search(root, “CA”)



# 3. Implementation - Search

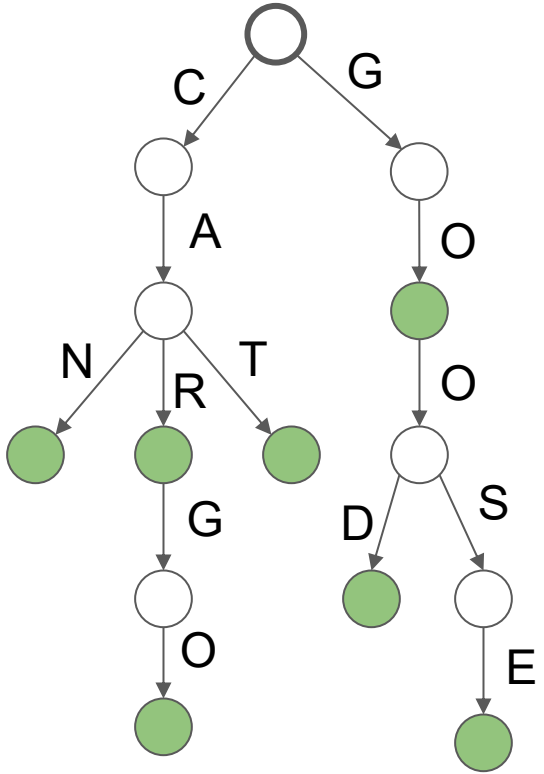
```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```



# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```

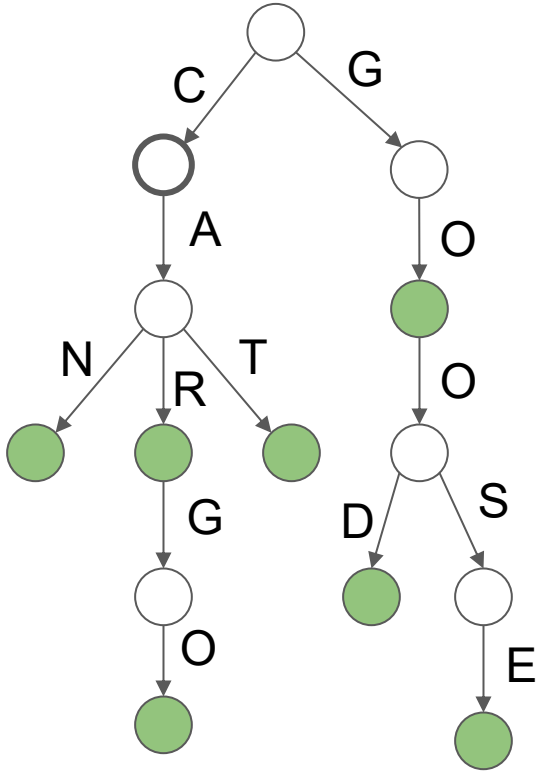
Has node at ‘C’



# 3. Implementation - Search

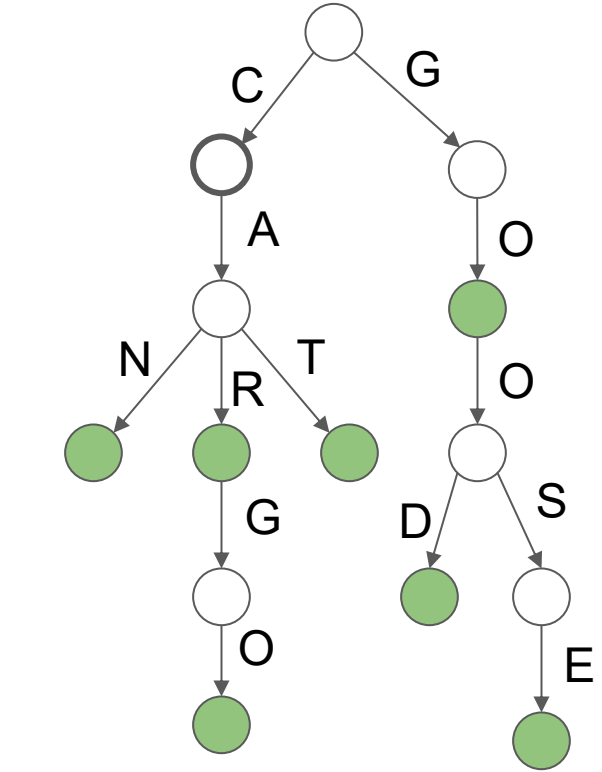
```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```



# 3. Implementation - Search

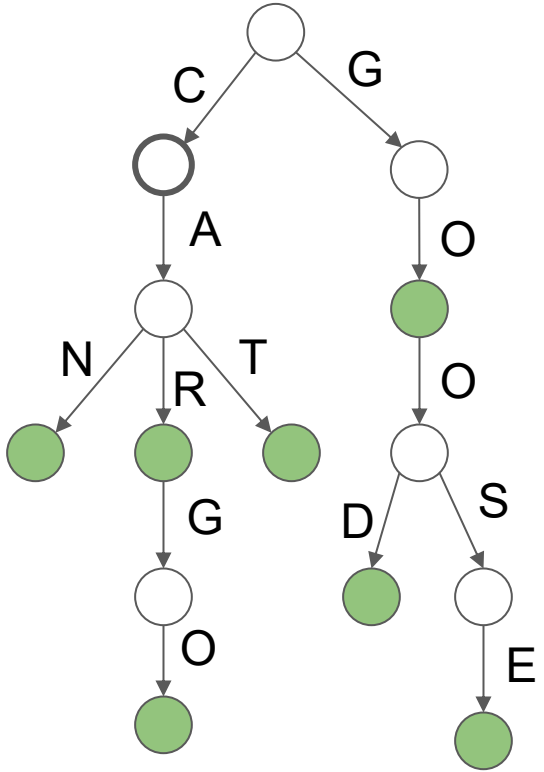
```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```



# 3. Implementation - Search

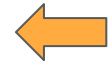
```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```

Has node at ‘A’

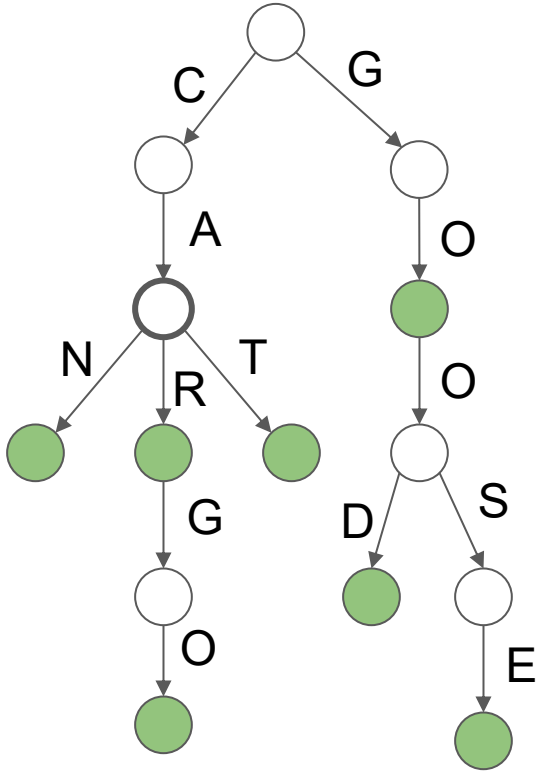


# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```



Go to child

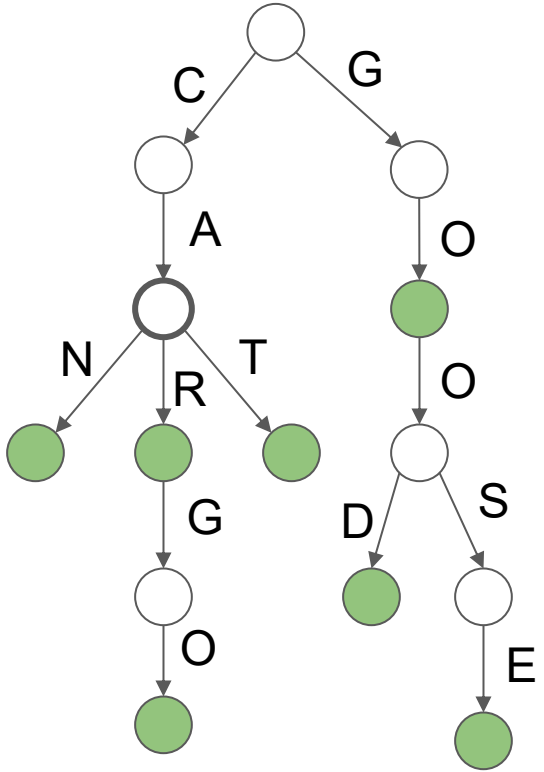


# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```



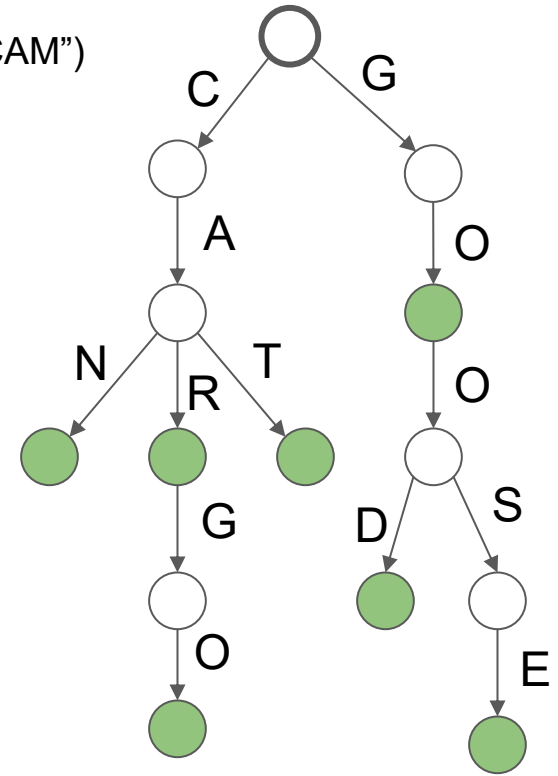
False



# 3. Implementation - Search

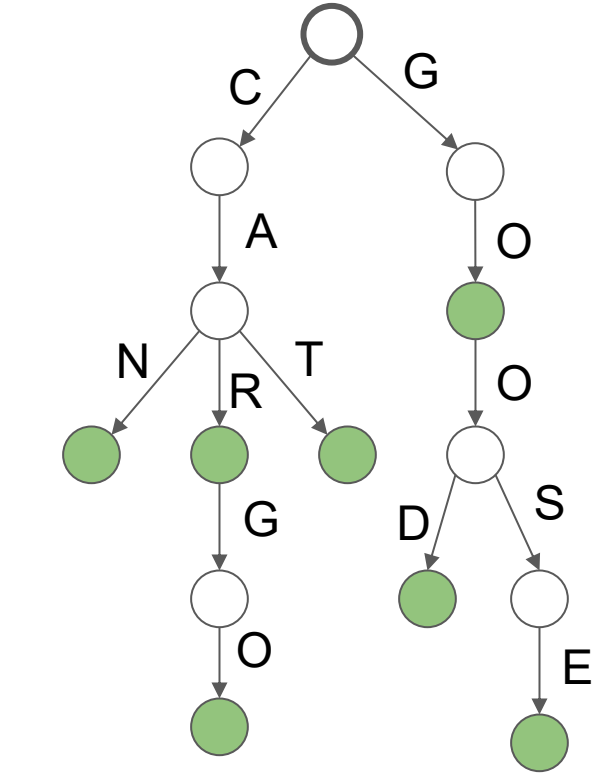
```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```

search(root, “CAM”)



# 3. Implementation - Search

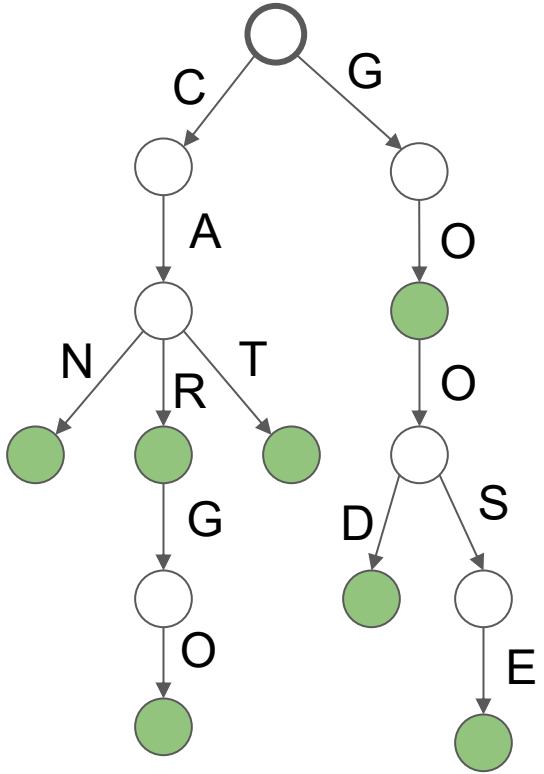
```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```



# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```

Has node at ‘C’

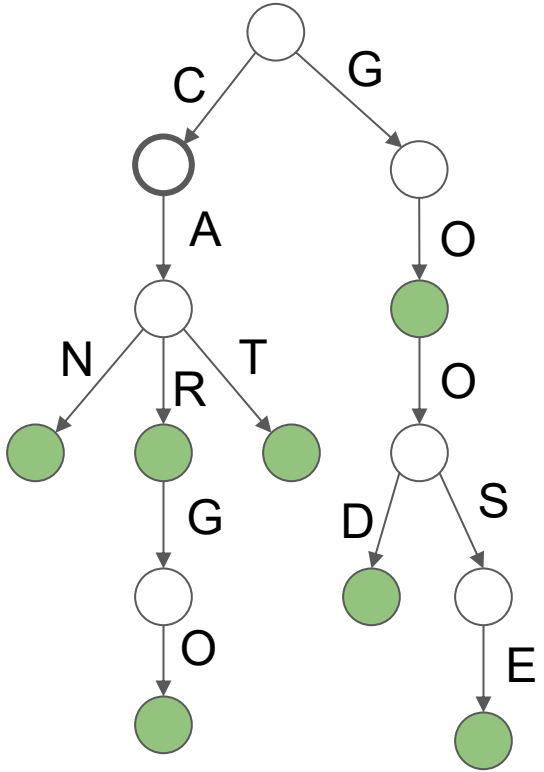


# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```



Go to child

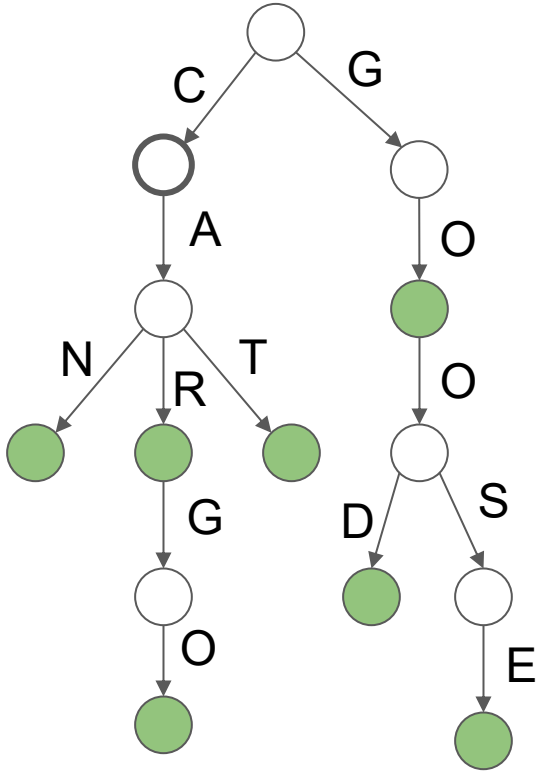


# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```



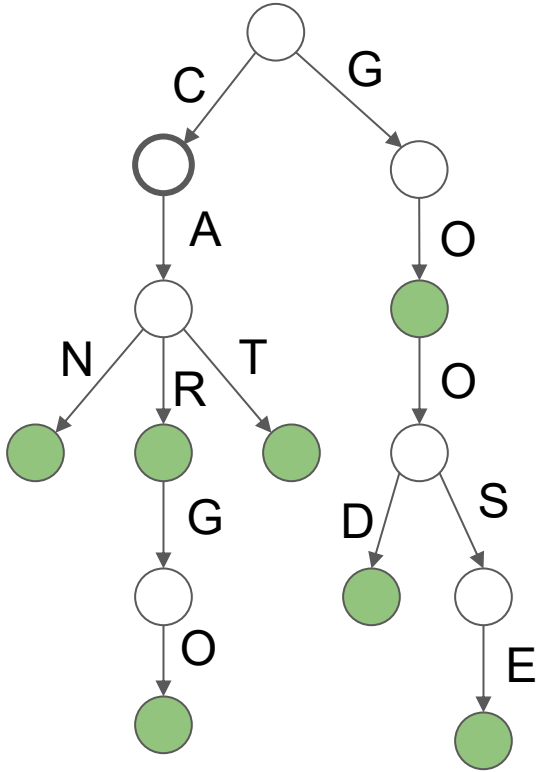
c = ‘A’



# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```

Has node at ‘A’

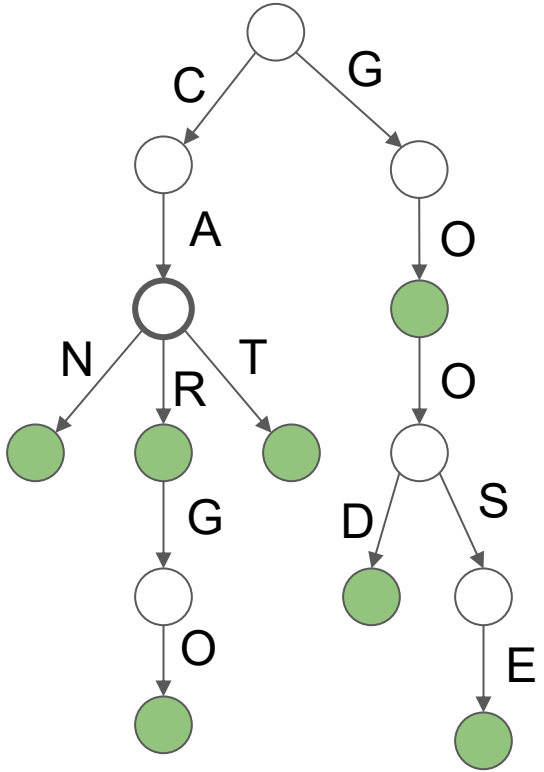


# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```



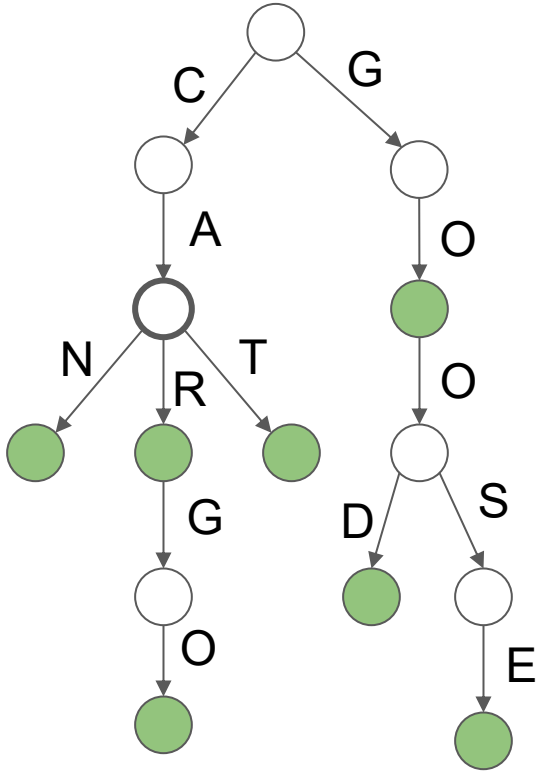
Go to child



# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```

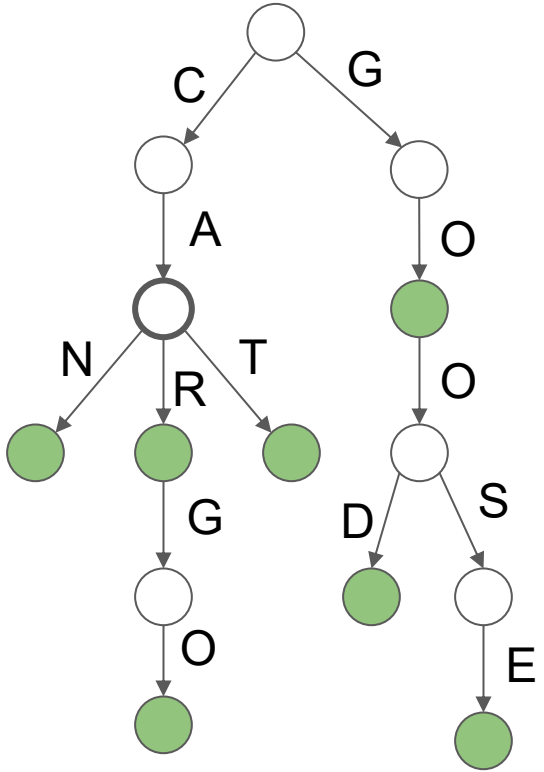


# 3. Implementation - Search

```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```

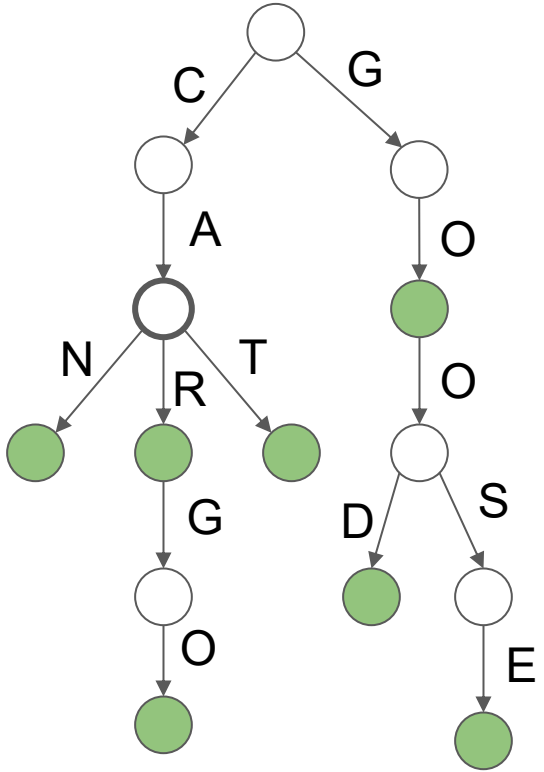
No node at ‘M’



# 3. Implementation - Search

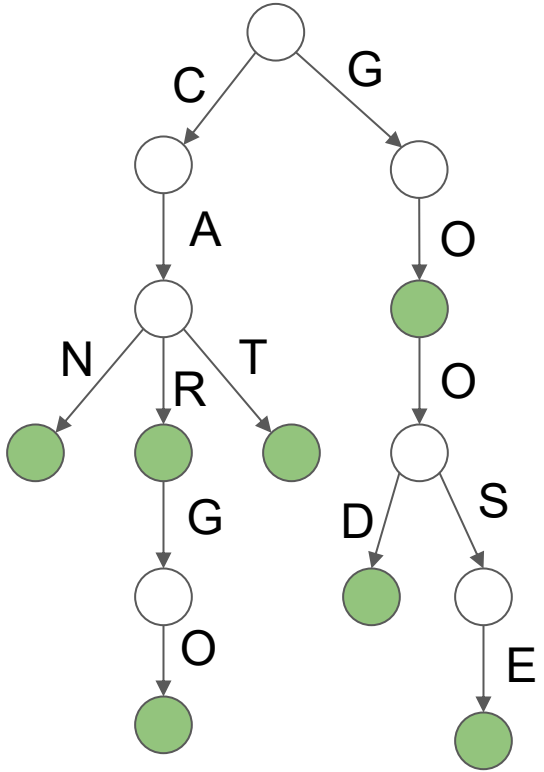
```
def search(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return False
    end if
    node = node.children[c]
  end for
  return node.is_terminal
```



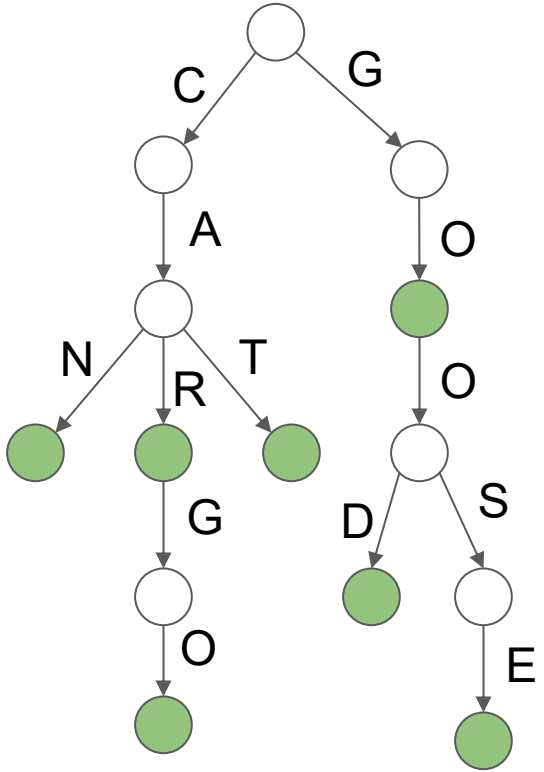
# 3. Implementation - Deletion

- Following the string within the trie like search.
- Assign is_terminal of the final node to False.



# 3. Implementation - Deletion

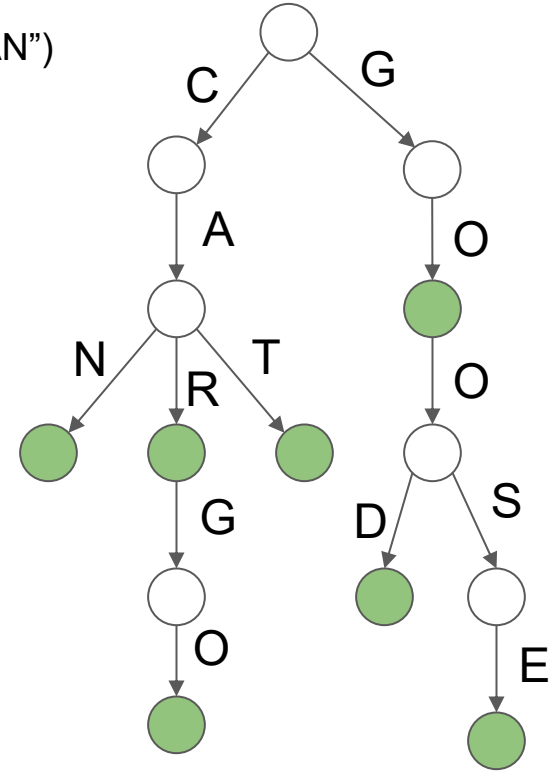
```
def delete(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return
    end if
    node = node.children[c]
  end for
  node.is_terminal = False
```



# 3. Implementation - Deletion

```
def delete(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return
    end if
    node = node.children[c]
  end for
  node.is_terminal = False
```

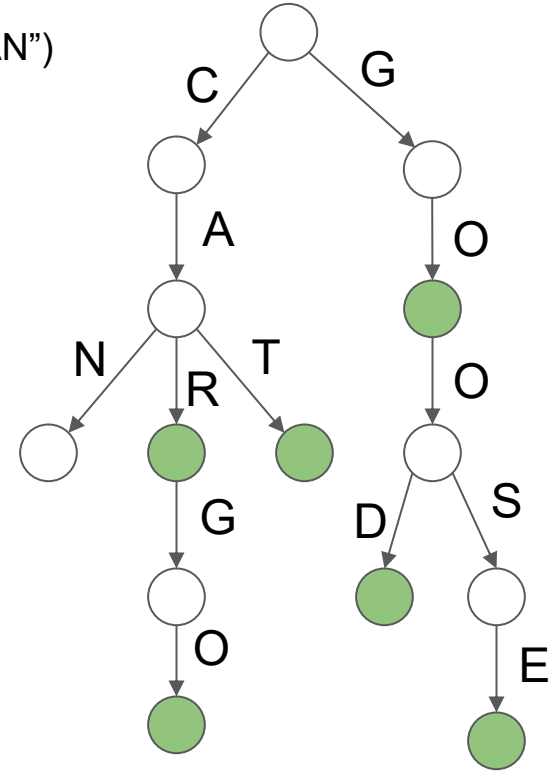
delete(root, “CAN”)



# 3. Implementation - Deletion

```
def delete(node: Node, s: String)
  for c in s:
    if node.children[c] == nil:
      return
    end if
    node = node.children[c]
  end for
  node.is_terminal = False
```

delete(root, “CAN”)



# II. Application

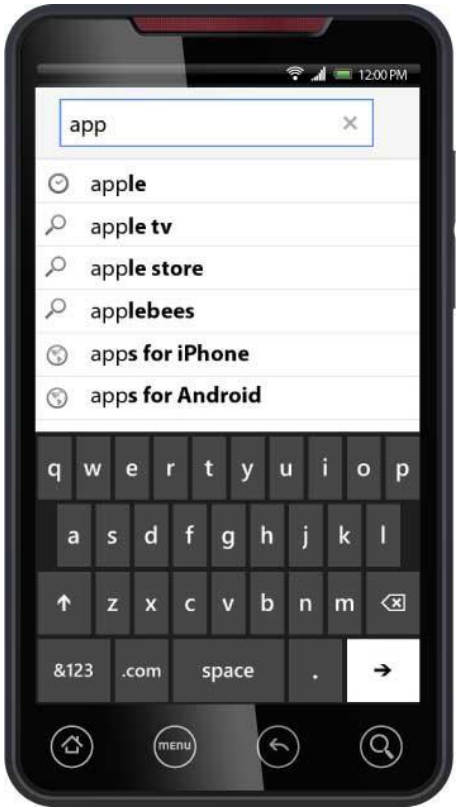
# 1. The most important property of a trie

- Trie is a data structure that can **insert**, **search**, **delete** a string S in **O(|S|)**.

- Which means if there are a lot of short strings, all operations will be very *fast*.
- Where do we find ourselves dealing with a lot of short strings ? Dictionaries !

# 2. Autocomplete for word

- Autocomplete for word is a feature on most smartphone nowadays.
- Also can be found in search engine, text editor, CLIs, …
- Trie is one of the earliest way to autocomplete.
- Variations of trie are used to tackle this problem nowadays.



# 2. Autocomplete for word

Problem:

- Given a dictionary of words.
- Each query is a prefix of word, output all words start with this prefix.

Example:

- Input: “ho”
- Output: [‘horse’, ‘house’, ‘hour’, ‘hourglass’, … ]

# 2. Autocomplete for word - Solution

Solution:

- Build a trie for the dictionary.
- With each input, travel to the last node of last character, then do a DFS from this node on trie and output all word.

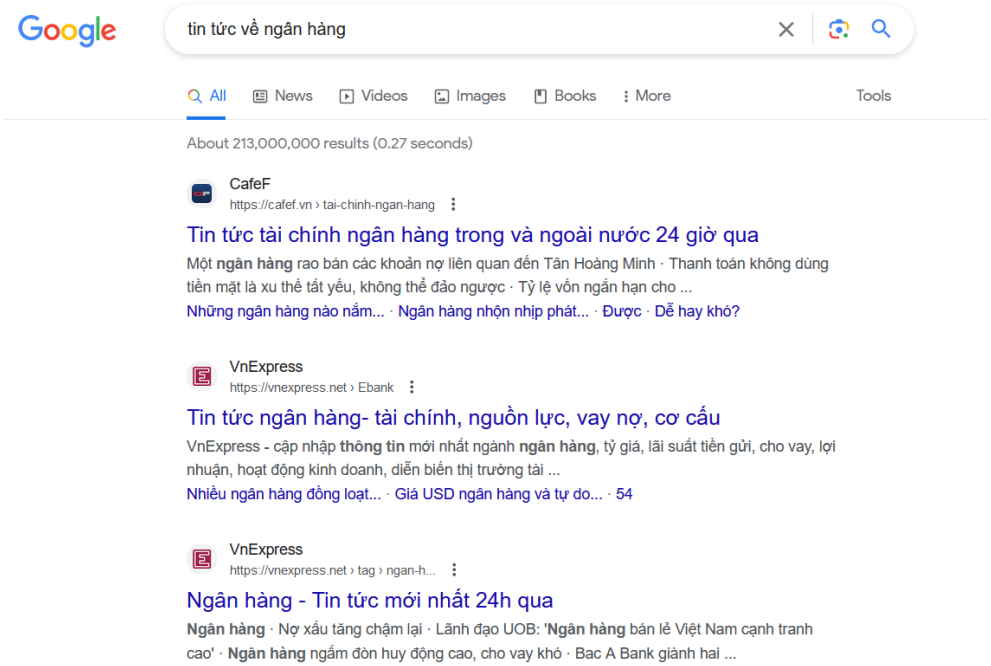
# 2. Autocomplete for word - Solution

Challenges for the listener:

- Rank output words:.
  - Example: “ho” -> “house” should be standing before “hourglass” in most cases.
  - “Zebra is a ho” -> should recommend “horse” as the first option.
- Update ranking based on user’s behavior.
- Autocorrect for spelling mistake.

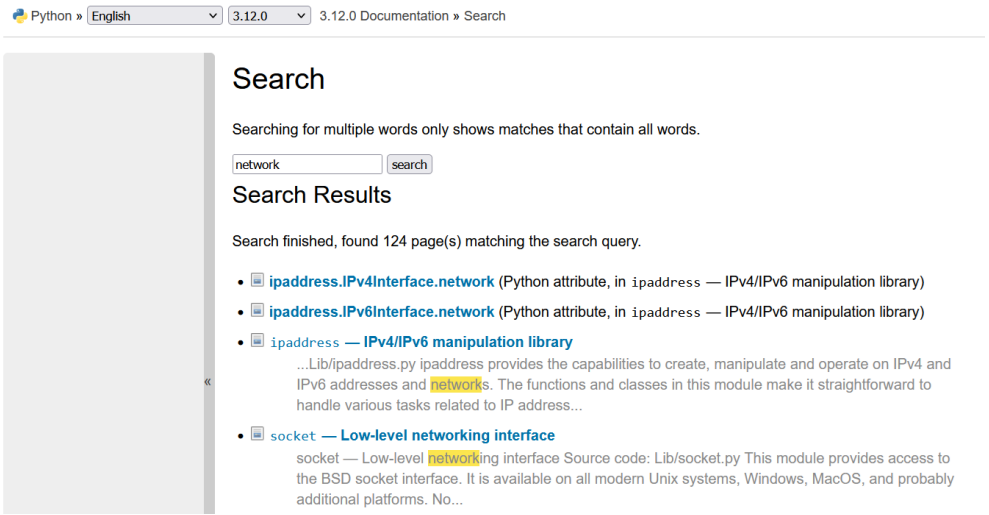
# 3. Searching document with keywords

- We type keywords into Google, and it returns a bunch of web pages that match those keywords.



# 3. Searching document with keywords

- We type keywords into the search bar of python documentation, and it returns a bunch of pages that match the keywords.



# 3. Searching document with keywords

Problem:

- Given a set of documents before hand.
- Each query contains some words.
- Output the list of document that has at least one of the words in each query.

Example:

- Input:
  - docs = [[‘nice day’], [‘nice house’], [‘pink house’]]
  - queries = [‘nice’, ‘house’, ‘nice pink’]
- Output: [[0, 1], [1, 2], [0, 1, 2]]

# 3. Searching document with keywords - Solution

Solution:

- Indexing the set of document by word.
- Add field **doc_indices** which is a set of indices for documents to trie node.
- With each document, with each word, insert that word to the trie and at terminal node, append document index to **doc_indices**.
- With each word in query, we know the **doc_indices** for this word.
- Union all **doc_indices**.

# 3. Searching document with keywords - Solution

Challenges for the listener:

- Ranking document: which document is more “important” ?
- More query types: AND, OR, EXACT, NOT, …
- Searching a phrase.

# III. Exercises

# 1. Splitting merge words

Problem:

- Given a dictionary and a string written continuously without space.
- Split the string to words.
- Limit:
  - len(dictionary) = T <= 3e4
  - max(len(word in dictionary)) = M = 100
  - len(string) = N <= 1e5

Example:

- Input:
  - Dictionary: {this, is, an, exam, example, string, ring}
  - String: “thisisanexamplestring”
- Output: “this is an example string”

# 1. Splitting merge words - Solution

Solution:

- Let f[i] = True if s[1..i] can be split into words, False otherwise.
- f[0] = True.
- f[i] = True if f[j] = True and s[j+1..i] is a word in dictionary.
- f[N] = True -> can split.
- s[j+1..i] is a word in dictionary -> using trie of dictionary to check
- N is the length of string
- M is the length of the longest words in dictionary.
- Dynamic programming takes O(N.M^2)
- Can you optimize it to O(N.M) ?

# 1. Splitting merge words - Solution

```
f = [0] * (n+1)
trace = [-1] * (n+1)
f[0] = True
for i in [1, 2, …, N]:
  for j in [N-M, N-M+1, …, i-1]:
    if (
      j >= 0
      and f[j]
      and search(root, s[j+1..i])
    ):
      f[i] = True
      tr[i] = j
```

```
u = N
while u != 0:
  words.append(s[trace[u]+1 .. u])
  u = tr[u]

words = reversed(words)
```

# 2. k-th smallest element

Problem:

- Design a data structure that can supports following operations:
  - Add a positive number (int32).
  - Find the k-th smallest element.
- Every operation should be done online.

Example:

- Input : [‘Add 4’, ‘Add 8’, ‘Add 7’, ‘Find 2’, ‘Find 1’, ‘Add 2’, ‘Find 2’]
- Output: [None, None, None, 7, 4, None, 4]

# 2. k-th smallest element - Solution

Solution:

- Change number to fixed 32-character binary string.
- Each node has count words for each links.
- When insert, add count along the way.
- When query for k-th smallest element, use count to know where to branch.
- Add and Find both have O(1) complexity.

# 2. k-th smallest element - Solution

```
Node:
  is_end: bool
  next: list[2]
  cnt: list[2] = [0, 0]

def insert(s):
  cur = root
  for c in s:
    if cur.next[c] == nil
      cur.next[c] = Node()
      cur.cnt[c] += 1
    cur = cur.next[c]
  cur.is_end = True
```

```
def search(k):
  cur = root
  res = 0
  while k > 0:
    for c in [0, 1]:
      if k > cur.cnt[c]:
        k -= cur.cnt[c]
      else:
        res = res*2 + c
        cur = cur.next[c]
        break
  return res
```

# 3. Lexical sorting

- Problem: Given an array of string, sort them in lexical order.
- Example:
  - Input: [‘c’, ‘b’, ‘a’, ‘an’, ‘bee’, ‘be’, ‘boo’]
  - Output: [‘a’, ‘an’, ‘b’, ‘be’, ‘bee’, ‘boo’, ‘c’]
- Assume N strings, each string is M characters long.
- Using normal sorting algo: O(M.N.logN) (Comparing 2 string takes O(M) ).
- Using trie ?

# 3. Lexical sorting - Solution

- Problem: Given an array of string, sort them in lexical order.
- Example:
  - Input: [‘c’, ‘b’, ‘a’, ‘an’, ‘bee’, ‘be’, ‘boo’]
  - Output: [‘a’, ‘an’, ‘b’, ‘be’, ‘bee’, ‘boo’, ‘c’]
- Assume N strings, each string is M characters long.
- Using normal sorting algo: O(M.N.logN) (Comparing 2 string takes O(M) ).
- Using trie ?
  - Build a trie for the array then traveling in pre-order: O(M.N)

Thank you for listening !

# Q & A

# More References

- VNOI Trie (Vietnamese)
- Autocomplete Algorithms
- More complex tries variation